

CSA0624 - Design and Analysis of Algorithms

Brute Force — Sorting Algorithms

Name: Aryan Arun

Reg.No: 192411157

1. Selection Sort

Question 1: Trophy Ranking System

Scenario: A sports event organizer has recorded the scores of 50 participants but needs to display only the top 5 scores on a leaderboard screen. Since only a few top values are needed (not a full sort), use selection sort's "find maximum repeatedly" approach to extract the top 5.

Approach: Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required, avoiding the cost of sorting all 50 values fully.

Complexity Analysis: $O(k \cdot n)$ instead of $O(n^2)$ for a full sort, where k is the number required (here $k=5$) and n is the total count — since only k passes are made instead of n , each still scanning the shrinking remainder.

Python Code:

```
def top_k_scores(scores, k):
    arr = scores.copy()
    n = len(arr)
    result = []
    limit = min(k, n)
    for i in range(limit):
        max_idx = i
        for j in range(i + 1, n):
            if arr[j] > arr[max_idx]:
                max_idx = j
        arr[i], arr[max_idx] = arr[max_idx], arr[i]
        result.append(arr[i])
    return result

print(top_k_scores([72,88,65,90,77,95,60,83,91,68], 5))
assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
assert top_k_scores([5,3,1], 5) == [5,3,1] # fewer than k items
assert top_k_scores([], 3) == [] # empty input
print('All test cases passed!')
```

Output:

```
[95, 91, 90, 88, 83]
All test cases passed!
```

Question 2: Memory-Constrained Embedded Device

Scenario: A low-memory IoT sensor device collects temperature readings and must sort them with minimal write/swap operations, since each write to memory is costly on this hardware. Selection sort performs at most $n-1$ swaps regardless of input order, making it preferable to bubble or insertion sort here.

Approach: Use the standard selection sort algorithm, but only perform a swap when the found minimum is not already in the correct position. Track the number of swaps performed and compare it against $n-1$ (the theoretical maximum) to demonstrate the low write cost.

Complexity Analysis: $O(n^2)$ comparisons, but only $O(n)$ swaps in the worst case (at most $n-1$) — the key property exploited here, since comparisons are cheap on this hardware while writes/swaps are the expensive operation.

Python Code:

```
def selection_sort_min_writes(arr):
    a = arr.copy()
    n = len(a)
    swaps = 0
    for i in range(n - 1):
        min_idx = i
```

```

        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            # only swap when actually needed
            a[i], a[min_idx] = a[min_idx], a[i]
            swaps += 1
    return a, swaps

res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
print(f"Sorted: {res}, Swaps: {sw}")
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1          # write bound holds

res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
print(f"Already sorted -> Swaps: {sw2}")
assert sw2 == 0                    # already sorted -> zero swaps
print('All test cases passed!')

```

Output:

```

Sorted: [18.8, 19.2, 21.4, 23.5, 25.1], Swaps: 2
Already sorted -> Swaps: 0
All test cases passed!

```

Question 3: Library Book Reordering

Scenario: A librarian wants to rearrange a shelf of 20 books by ID number, but physically moving a book (swap) is time-consuming, while checking labels (comparison) is fast. Model this as an array sorting problem using selection sort and report the number of physical "moves" needed.

Approach: Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are unrestricted since they are cheap.

Complexity Analysis: $O(n^2)$ label comparisons (cheap, unrestricted), but at most $n-1$ physical moves (expensive, minimized) — the same swap-minimizing property of Selection Sort as Question 2, applied to physical relocation cost.

Python Code:

```

def reorder_shelf(arr):
    a = arr.copy()
    n = len(a)
    moves = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            # only physically move when needed
            a[i], a[min_idx] = a[min_idx], a[i]
            moves += 1
    return a, moves

ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
print(f"Ordered: {ordered}, Moves: {moves}")
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
assert moves <= len(ordered) - 1

ordered2, moves2 = reorder_shelf([100, 200, 300])    # already sorted
print(f"Already sorted -> Moves: {moves2}")
assert moves2 == 0
print('All test cases passed!')

```

Output:

```
Ordered: [101, 102, 118, 199, 250, 305, 400], Moves: 4
Already sorted -> Moves: 0
All test cases passed!
```

Question 4: Contest Prize Distribution

Scenario: In a coding contest, you must repeatedly select the participant with the current highest score, print their name, remove them from the list, and repeat until all participants are ranked. Implement this using selection sort logic without using a separate sorting library.

Approach: Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as the next-ranked winner.

Complexity Analysis: $O(n^2)$ — identical structure to standard Selection Sort, just scanning for the maximum instead of the minimum at each of the n positions.

Python Code:

```
def distribute_prizes(participants):
    arr = list(participants)
    n = len(arr)
    ranking = []
    for i in range(n):
        max_idx = i
        for j in range(i + 1, n):
            if arr[j][1] > arr[max_idx][1]:
                max_idx = j
        arr[i], arr[max_idx] = arr[max_idx], arr[i]
        ranking.append(arr[i])
        print(f"Rank {i+1}: {arr[i][0]} with score {arr[i][1]}")
    return ranking

ranking = distribute_prizes([('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)])
scores_only = [p[1] for p in ranking]
assert scores_only == sorted(scores_only, reverse=True) # descending order
assert ranking[0][1] == 95                             # top score first
print('All test cases passed!')
```

Output:

```
Rank 1: Ravi with score 95
Rank 2: Dev with score 95
Rank 3: Asha with score 88
Rank 4: Meera with score 79
All test cases passed!
```

Question 5: Small Dataset in a Mobile App

Scenario: A shopping app needs to sort a "recently viewed" list of just 8–10 items by price whenever the user opens the screen. Argue whether selection sort is a reasonable choice for such a small, frequently-refreshed list, and implement it with timing to support your answer.

Approach: For very small n , the $O(n^2)$ cost of selection sort is negligible in absolute terms (a handful of microseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain choice rather than pulling in a general-purpose sort for 8–10 items.

Complexity Analysis: $O(n^2)$, but with $n \approx 8$ –10 this is at most ~ 45 comparisons — measured here in microseconds, confirming that the asymptotic inefficiency is irrelevant at this scale.

Python Code:

```
import time
```

```
def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
    return a

data = [499,129,899,45,275,60,310,150]
start = time.perf_counter()
result = selection_sort(data)
elapsed = time.perf_counter() - start
print(f"Sorted: {result}")
print(f"Time taken: {elapsed*1e6:.2f} microseconds")

assert selection_sort([499,129,899,45,275,60,310,150]) ==
sorted([499,129,899,45,275,60,310,150])
assert selection_sort([]) == []           # empty list
assert selection_sort([7]) == [7]        # single item
print('All test cases passed!')
```

Output:

```
Sorted: [45, 60, 129, 150, 275, 310, 499, 899]
Time taken: 6.82 microseconds
All test cases passed!
```

2. Bubble Sort

Question 1: Exam Result Slip Correction

Scenario: A teacher has a nearly sorted list of student roll numbers (only 2–3 are out of place due to late corrections). Use optimized bubble sort (with an early-exit flag) to fix the order and show it takes very few passes.

Approach: Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass completes with no swaps, the list is already sorted and the algorithm exits early — ideal for nearly sorted data like this.

Complexity Analysis: Best case (nearly sorted, early exit): close to $O(n)$. Worst case (no early exit possible): $O(n^2)$. Here, only 2 passes were needed instead of the full 8.

Python Code:

```
def optimized_bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    passes = 0
    for i in range(n):
        passes += 1
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            # no swaps -> already sorted, exit early
            break
    return a, passes

sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])
print(f"Sorted: {sorted_rolls}, Passes: {passes}")
assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])
assert passes < 8
    # fewer than full n passes

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5])
print(f"Already sorted -> Passes: {passes_ok}")
assert passes_ok == 1
    # exits after first pass
print('All test cases passed!')
```

Output:

```
Sorted: [101, 102, 103, 104, 105, 106, 107, 108], Passes: 2
Already sorted -> Passes: 1
All test cases passed!
```

Question 2: Traffic Signal Priority Queue

Scenario: At an intersection, vehicle priorities (ambulance > bus > car) arrive in a small queue and need constant reordering as new vehicles arrive. Simulate this using bubble sort each time a new vehicle is added, and discuss whether bubble sort's simplicity outweighs its inefficiency here.

Approach: Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's $O(n^2)$ cost is negligible, and its simplicity makes it easy to reason about and verify in a safety-critical system.

Complexity Analysis: $O(n^2)$ in general, but with a small, bounded queue size n , this reduces to a small constant number of operations — the simplicity and predictability of Bubble Sort outweigh its inefficiency at this scale, which matters for verifiability in safety-critical logic.

Python Code:

```
def bubble_sort_queue(queue):
    priority = {'ambulance': 3, 'bus': 2, 'car': 1}
    arr = queue.copy()
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if priority[arr[j]] < priority[arr[j + 1]]: # descending priority
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
print(f"Queue: {queue} -> Ordered: {result}")
assert result == ['ambulance', 'bus', 'car', 'car']

assert bubble_sort_queue(['ambulance']) == ['ambulance'] # single vehicle
print('All test cases passed!')
```

Output:

```
Queue: ['car', 'car', 'bus', 'ambulance'] -> Ordered: ['ambulance', 'bus', 'car', 'car']
All test cases passed!
```

Question 3: Bubble Sort Visualization Tool

Scenario: You are building an educational tool to visually demonstrate how "bubbles" (largest elements) rise to the top with each pass. Implement bubble sort so it prints/returns the state of the array after every single pass, useful for animation.

Approach: Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through UI.

Complexity Analysis: $O(n^2)$ time (unchanged from standard Bubble Sort) plus $O(n)$ extra space to store the $n+1$ frames — a small, worthwhile overhead for visualization purposes.

Python Code:

```
def bubble_sort_with_frames(arr):
    a = arr.copy()
    n = len(a)
    frames = [a.copy()] # frame 0: original input
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        frames.append(a.copy()) # frame after this pass
        if not swapped:
            break
    return frames

frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
for idx, f in enumerate(frames):
    print(f"Pass {idx}: {f}")

assert frames[-1] == sorted([5, 1, 4, 2, 8]) # final frame is sorted
assert frames[0] == [5, 1, 4, 2, 8] # first frame is the original input
assert len(frames) >= 2 # at least an initial and final frame
print('All test cases passed!')
```

Output:

```
Pass 0: [5, 1, 4, 2, 8]
Pass 1: [1, 4, 2, 5, 8]
Pass 2: [1, 2, 4, 5, 8]
Pass 3: [1, 2, 4, 5, 8]
All test cases passed!
```

Question 4: Sorting Sensor Alerts by Severity

Scenario: A monitoring dashboard receives 15 alerts per minute that must be sorted by severity level (1 = critical to 5 = low) for display. Use bubble sort and measure whether the early-termination optimization meaningfully improves performance on this small, frequently-updated dataset.

Approach: Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the optimization helps for this small, often-partially-sorted stream.

Complexity Analysis: Plain Bubble Sort: always $O(n^2)$ comparisons regardless of order. Optimized Bubble Sort: same $O(n^2)$ worst case, but fewer comparisons in practice whenever an early sorted state is reached — confirmed numerically below (99 vs 105 comparisons).

Python Code:

```
def bubble_sort_plain(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n):
        for j in range(0, n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a, comparisons

def bubble_sort_optimized(arr):
    a = arr.copy()
    n = len(a)
    comparisons = 0
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a, comparisons

alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)
assert r1 == r2 == sorted(alerts)      # both produce the same sorted result
assert c2 <= c1                        # optimized never does more comparisons
print('Plain comparisons:', c1, '| Optimized comparisons:', c2)
print('All test cases passed!')
```

Output:

```
Plain comparisons: 105 | Optimized comparisons: 99
All test cases passed!
```

Question 5: Card Game Hand Sorting

Scenario: In a card game app, a player's hand of 13 cards must be sorted by rank whenever a new card is drawn. Since only one card is added at a time (nearly sorted data), implement bubble sort and compare its pass count to a full re-sort from scratch.

Approach: Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel) compared to sorting the entire hand from an unsorted state.

Complexity Analysis: Nearly-sorted (one card added): close to $O(n)$ passes, proportional to the new card's displacement. Fully shuffled: up to $O(n)$ passes as well in the worst case, but empirically more than the nearly-sorted case, as confirmed below.

Python Code:

```
import random

def bubble_sort_hand(arr):
    a = arr.copy()
    n = len(a)
    passes = 0
    for i in range(n):
        passes += 1
        swapped = False
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a, passes

random.seed(42)
hand = [2, 4, 6, 8, 9, 11, 13]
hand.append(7)  # nearly sorted: one new card
final_hand, passes_incremental = bubble_sort_hand(hand)
assert final_hand == sorted(hand)

shuffled = hand.copy()
random.shuffle(shuffled)
_, passes_full = bubble_sort_hand(shuffled)
print(f"Nearly-sorted hand: {[2,4,6,8,9,11,13,7]} -> {passes_incremental} passes")
print(f"Shuffled hand: {shuffled} -> {passes_full} passes")
assert passes_incremental <= passes_full  # nearly-sorted needs no more passes
print('All test cases passed!')
```

Output:

```
Nearly-sorted hand: [2, 4, 6, 8, 9, 11, 13, 7] -> 5 passes
Shuffled hand: [8, 9, 13, 7, 6, 11, 2, 4] -> 7 passes
All test cases passed!
```

3. Insertion Sort

Question 1: Live Leaderboard Updates

Scenario: An online game updates a leaderboard every time a single player's score changes, while the rest of the list remains sorted. Use insertion sort to insert just the updated score into its correct position instead of re-sorting the entire list, and measure the efficiency gain.

Approach: Use the insertion-sort technique of shifting elements (here, shifting smaller scores rightward, since the board is sorted in descending order) to find and insert the new score into its correct position in the already-sorted leaderboard, avoiding a full re-sort.

Complexity Analysis: $O(n)$ worst case (shifting past all existing entries) instead of $O(n \log n)$ for re-sorting the whole board from scratch — a substantial saving when only one value changes at a time.

Python Code:

```
def insert_updated_score(board, new_score):
    arr = board.copy()
    arr.append(None)          # make room for the new score
    n = len(arr)
    j = n - 2
    shifts = 0
    while j >= 0 and arr[j] < new_score:  # board is descending: shift smaller scores right
        arr[j + 1] = arr[j]
        j -= 1
        shifts += 1
    arr[j + 1] = new_score
    return arr, shifts

board = [980, 875, 760, 690, 500]
updated_board, shifts = insert_updated_score(board, 820)
print(f"Board after inserting 820: {updated_board}, Shifts: {shifts}")
assert updated_board == [980, 875, 820, 760, 690, 500]

board2, shifts2 = insert_updated_score(board, 100)  # lowest score
print(f"Board after inserting 100: {board2}, Shifts: {shifts2}")
assert board2[-1] == 100 and shifts2 == 0  # no shifts needed, goes to the end
print('All test cases passed!')
```

Output:

```
Board after inserting 820: [980, 875, 820, 760, 690, 500], Shifts: 3
Board after inserting 100: [980, 875, 760, 690, 500, 100], Shifts: 0
All test cases passed!
```

Question 2: Card Sorting While Playing

Scenario: A card player sorts their hand as they pick up cards one at a time (the classic real-world analogy for insertion sort). Implement this behavior programmatically, sorting a growing hand card by card.

Approach: Maintain a hand list. Each time a new card is picked up, insert it into its correct position among the already-sorted cards by shifting larger cards one position to the right, exactly like insertion sort's inner loop.

Complexity Analysis: $O(n)$ per card picked up (worst case), giving $O(n^2)$ total for building a hand of n cards one at a time — matching the natural process of a human player sorting cards as they are dealt.

Python Code:

```
def pick_up_card(hand, card):
    arr = hand.copy()
    arr.append(card)
    j = len(arr) - 2
    while j >= 0 and arr[j] > card:      # shift larger cards right
```

```

        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = card
    return arr

hand = []
for card in [7, 2, 9, 4, 1]:
    hand = pick_up_card(hand, card)
    print(f"Picked up {card} -> hand: {hand}")

assert hand == sorted([7, 2, 9, 4, 1])
assert pick_up_card([], 5) == [5]      # first card picked up
print('All test cases passed!')

```

Output:

```

Picked up 7 -> hand: [7]
Picked up 2 -> hand: [2, 7]
Picked up 9 -> hand: [2, 7, 9]
Picked up 4 -> hand: [2, 4, 7, 9]
Picked up 1 -> hand: [1, 2, 4, 7, 9]
All test cases passed!

```

Question 3: Real-Time Stock Price Feed

Scenario: A trading dashboard receives stock prices one at a time and must maintain a sorted list at all times for quick min/max lookups. Use insertion sort to insert each new incoming price efficiently into the already-sorted list.

Approach: Keep a running sorted list of prices. For each incoming price, use binary search to find its correct position in $O(\log n)$, then insert it — the core idea of insertion sort applied incrementally, with a faster position lookup.

Complexity Analysis: $O(\log n)$ to locate the position via binary search, plus $O(n)$ worst case to shift/insert elements in a list — the search itself is logarithmic, though the underlying list insertion is still linear due to the need to shift elements.

Python Code:

```

import bisect

def insert_price(prices, p):
    arr = prices.copy()
    pos = bisect.bisect_left(arr, p)    #  $O(\log n)$  binary search for position
    arr.insert(pos, p)
    return arr

prices = []
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
    prices = insert_price(prices, p)
    print(f"Inserted {p} -> prices: {prices}")

assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])
assert prices[0] == min(prices) and prices[-1] == max(prices)    #  $O(1)$  min/max lookup
print('All test cases passed!')

```

Output:

```

Inserted 102.5 -> prices: [102.5]
Inserted 98.3 -> prices: [98.3, 102.5]
Inserted 105.1 -> prices: [98.3, 102.5, 105.1]
Inserted 100.0 -> prices: [98.3, 100.0, 102.5, 105.1]
Inserted 97.8 -> prices: [97.8, 98.3, 100.0, 102.5, 105.1]
All test cases passed!

```

Question 4: Playlist Reordering by Recently Added

Scenario: A music app lets users insert a newly added song into a playlist that is already sorted by duration. Implement insertion sort logic to place just the new song correctly instead of sorting the whole playlist again.

Approach: Append the new song's duration to the playlist and shift it leftward past any longer songs until it reaches its correct sorted position, exactly as the inner loop of insertion sort does for a single new element.

Complexity Analysis: $O(n)$ worst case to shift past all existing songs — far cheaper than an $O(n \log n)$ full re-sort of the whole playlist for a single insertion.

Python Code:

```
def insert_song(playlist, song):
    arr = playlist.copy()
    arr.append(song)
    j = len(arr) - 2
    while j >= 0 and arr[j][1] > song[1]:    # compare by duration
        arr[j + 1] = arr[j]
        j -= 1
    arr[j + 1] = song
    return arr

playlist = [('Intro', 120), ('Chill Beat', 210), ('Long Jam', 340)]
updated_playlist = insert_song(playlist, ('Quick Track', 180))
print(f"Updated playlist: {updated_playlist}")

durations = [s[1] for s in updated_playlist]
assert durations == sorted([120, 210, 340, 180])
assert ('Quick Track', 180) in updated_playlist
print('All test cases passed!')
```

Output:

```
Updated playlist: [('Intro', 120), ('Quick Track', 180), ('Chill Beat', 210), ('Long Jam',
340)]
All test cases passed!
```

Question 5: Nearly Sorted Sensor Log Correction

Scenario: A weather station logs temperature readings in mostly chronological order, but occasional delayed transmissions cause a few entries to be out of place. Use insertion sort to fix the log and show that the number of shifts is very low compared to a randomly shuffled version of the same data.

Approach: Run standard insertion sort on the log and count total shifts performed. Because only a few entries are out of place, most elements require zero or one shift, resulting in a shift count close to the best-case $O(n)$ rather than the worst-case $O(n^2)$.

Complexity Analysis: Nearly-sorted input: close to $O(n)$ shifts (few elements out of place). Randomly shuffled input: closer to $O(n^2)$ shifts in the worst case — confirmed below (5 shifts nearly-sorted vs 15 shifts random, out of a possible 21 for $n=7$).

Python Code:

```
import random

def insertion_sort_count_shifts(arr):
    a = arr.copy()
    n = len(a)
    shifts = 0
    for i in range(1, n):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
```

```
        a[j + 1] = a[j]
        j -= 1
        shifts += 1
    a[j + 1] = key
    return a, shifts
```

```
random.seed(7)
log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]
sorted_log, shifts_nearly = insertion_sort_count_shifts(log)
assert sorted_log == sorted(log)

shuffled_log = log.copy()
random.shuffle(shuffled_log)
_, shifts_random = insertion_sort_count_shifts(shuffled_log)
print('Nearly-sorted shifts:', shifts_nearly, '| Random shifts:', shifts_random)
print('All test cases passed!')
```

Output:

```
Nearly-sorted shifts: 5 | Random shifts: 15
All test cases passed!
```